

Московский Авиационный Институт
(Национальный Исследовательский Университет)

Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу
«Операционные системы»

Студент: Бойцов Иван Алексеевич

Группа: М8О–212Б–22

Вариант: 5

Преподаватель: Миронов Евгений Сергеевич

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2024

Цель работы

Целью является приобретение практических навыков в:

1. Создание динамических библиотек;
2. Создание программ, которые используют функции динамических библиотек.

Задание

Требуется создать динамические библиотеки, которые реализуют определенный функционал. Далее использовать данные библиотеки 2-мя способами:

1. Во время компиляции (на этапе «линковки»/linking)
2. Во время исполнения программы. Библиотеки загружаются в память с помощью интерфейса ОС для работы с динамическими библиотеками

В конечном итоге, в лабораторной работе необходимо получить следующие части:

1. Динамические библиотеки, реализующие контракты, которые заданы вариантом;
2. Тестовая программа (программа №1), которая использует одну из библиотек, используя знания полученные на этапе компиляции;
3. Тестовая программа (программа №2), которая загружает библиотеки, используя только их местоположение и контракты.

Провести анализ двух типов использования библиотек. Пользовательский ввод для обеих программ должен быть организован следующим образом:

1. Если пользователь вводит команду «0», то программа переключает одну реализацию контрактов на другую (необходимо только для программы №2). Можно реализовать лабораторную работу без данной функции, но максимальная оценка в этом случае будет «хорошо»;
2. «1 arg1 arg2 ... argN», где после «1» идут аргументы для первой функции, предусмотренной контрактами. После ввода команды происходит вызов первой функции, и на экране появляется результат её выполнения;
3. «2 arg1 arg2 ... argM», где после «2» идут аргументы для второй функции, предусмотренной контрактами. После ввода команды происходит вызов второй функции, и на экране появляется результат её выполнения.

Контракты и реализации функций

№	Описание	Сигнатура	Реализация 1	Реализация 2
1	Расчет интеграла функции $\sin(x)$ на отрезке $[A, B]$ с шагом e	Float SinIntegral(float A, float B, float e)	Подсчет интеграла методом прямоугольников.	Подсчет интеграла методом трапеций.
6	Расчет значения числа e (основание натурального логарифма)	Float E(int x)	$(1+1/x)^x$	Сумма ряда по n от 0 до x , где элементы равны: $(1/(n!))$

Решение

Для выполнения задачи были созданы две динамические библиотеки Integral.dll и Eps.dll, содержащие функции для вычисления интегралов и значения числа e соответственно. Каждая из библиотек имеет две реализации: первая и вторая. В проекте реализованы два способа использования этих библиотек: на этапе компиляции (линковки) и на этапе выполнения.

1. **Динамическая библиотека Integral.dll** — содержит две реализации для вычисления интеграла функции $\sin(x)$ на заданном отрезке $[A, B]$:
 - IntegrateFirst — реализует метод прямоугольников;
 - IntegrateSecond — реализует метод трапеций.
2. **Динамическая библиотека Eps.dll** — содержит две реализации для вычисления числа e :
 - EpsFirst — использует формулу $((1 + \frac{1}{x})^x)$;
 - EpsSecond — использует разложение e в ряд Тейлора по формуле $(\sum_{n=0}^x \frac{1}{n!})$.

Обе библиотеки могут быть подключены на этапе компиляции (путем явного импорта) и на этапе выполнения (путем загрузки DLL с помощью функций ОС).

Описание программ

1. Программа с линковкой на этапе компиляции

Программа LinkingMain.cpp подключает библиотеки на этапе компиляции, используя заголовочные файлы Integral.h и Eps.h. При запуске пользователю предлагается выбрать библиотеку и метод, которые он хочет использовать. Программа поддерживает переключение между двумя реализациями функций:

- При выборе библиотеки integral пользователь вводит границы интегрирования (A) и (B) и шаг (ϵ). Программа считает интеграл с использованием выбранного метода.
- При выборе библиотеки eps пользователь вводит значение (x) для вычисления числа e, и программа выводит результат, используя выбранный метод.

2. Программа с линковкой на этапе выполнения

Программа DynamicMain.cpp загружает библиотеки в память во время выполнения. Это позволяет менять подключаемую библиотеку и реализацию функции без перекомпиляции кода.

Алгоритм работы следующий:

1. Пользователь выбирает библиотеку (integral или eps) и указывает, хочет ли он переключить реализацию.
2. На основании выбора загружается соответствующая библиотека и вызывается функция:
 - Для библиотеки integral вызывается метод прямоугольников (IntegrateFirst) или метод трапеций (IntegrateSecond);
 - Для библиотеки eps вызывается метод $((1 + \frac{1}{x})^x)$ (EpsFirst) или метод ряда Тейлора (EpsSecond).
3. Программа позволяет менять реализацию по ходу работы, что делает её более гибкой.

Код

Код, описанный ниже, реализует указанные функции и логику работы обеих программ. Код разбит на заголовочные и исходные файлы для динамических библиотек и программы, что способствует модульности.

Динамические библиотечки, указанные в задании, и их функции

Eps

```
// Расчет значения числа e(основание натурального логарифма)
#pragma once
```

```

extern "C" {
    __declspec(dllexport) float EpsFirst(int x);

    __declspec(dllexport) float EpsSecond(int x);
}
#include <iostream>
#include <cmath>
#include "Eps.h"

using namespace std;

extern "C" {
    // Вычисление факториала
    long long fact(int n) {
        long long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    //  $(1 + 1/x)^x$ 
    __declspec(dllexport) float EpsFirst(int x) {
        return pow((1 + 1.0 / (float)x), x);
    }

    // Сумма ряда по n от 0 до x, где элементы ряда равны:  $(1/(n!))$ 
    __declspec(dllexport) float EpsSecond(int x) {

        float to_return = 0;
        for (int n = 0; n < x; n++) {
            to_return = ((float)1 / fact(n));
        }

        return to_return;
    }
}

```

Integral

```

#pragma once
extern "C" {
    __declspec(dllexport) double IntegrateFirst(double a, double b, double eps);
}

```

```

    __declspec(dllexport) double IntegrateSecond(double a, double b, double eps);
}
#include <iostream>
#include <cmath>
#include "Integral.h"

using namespace std;
extern "C"{
    // Подсчёт интеграла методом прямоугольников
    __declspec(dllexport) double IntegrateFirst(double a, double b, double
eps) {

        int steps = fabs(b - a) / eps;
        double point = a;
        double result = 0;

        for (int i = 0; i < steps; i++) {
            result += sin(point) * eps;
            point += eps;
        }

        return result;
    }

    // Подсчёт интеграла методом трапеции
    __declspec(dllexport) double IntegrateSecond(float a, float b, float eps) {

        int steps = (b - a) / eps;
        double point = a;
        double result = 0;

        for (int i = 0; i < steps; i++) {
            result += sin(point + eps / 2) * eps;
            point += eps;
        }

        return result;
    }
}

```

Задание 1

DynamicMain

```
#include <iostream>
#include <windows.h>
#include <string>

using namespace std;

int main() {
    cout << "Программа join на этапе выполнения" << endl << endl;

    unsigned int switch_ = 1;

    while (true) {
        cout << "Введите название библиотеки, которую хотите подключить:
\n'integral' - Интеграл\n'eps' - Вычисление значения натурального
логарифма\n" << endl;
        cout << "Введите Y, если хотите переключить реализацию расчёта.
Изначальная реализация - 1" << endl;
        cout << "Введите 'exit' если хотите выйти из программы" << endl;

        string lib;
        string indicator;

        if (!(cin >> lib) || !(cin >> indicator)) {
            cout << "Завершение работы программы" << endl;
            break;
        } else if (indicator == "Y") {
            switch_ = (switch_ == 1) ? 2 : 1;
            cout << "Реализация переключена на " << switch_ << endl;
        }

        if (lib == "integral") {

            HMODULE hLib = LoadLibrary(L"Integral.dll");
            if (!hLib) {
                cerr << "Не удалось загрузить Integral.dll" << endl;
                continue;
            }

            auto IntegrateFirst = (double (*)(double, double, double))
GetProcAddress(hLib, "IntegrateFirst");
            auto IntegrateSecond = (double (*)(double, double, double))
GetProcAddress(hLib, "IntegrateSecond");
```

```

if (!IntegrateFirst || !IntegrateSecond) {
    cerr << "Ошибка загрузки функций из Integral.dll" << endl;
    FreeLibrary(hLib);
    continue;
}

double a, b, eps;
cout << "Введите границы отрезка интегрирования (a, b): " << endl;
cin >> a >> b;

cout << "Введите эпсилон: " << endl;
cin >> eps;

double result = (switch_ == 1) ? IntegrateFirst(a, b, eps) :
IntegrateSecond(a, b, eps);
cout << "Ответ: " << result << " - интеграл решен методом "
    << ((switch_ == 1) ? "прямоугольников" : "трапеций") << endl;

FreeLibrary(hLib);

} else if (lib == "eps") {

    HMODULE hLib = LoadLibrary(L"Eps.dll");
    if (!hLib) {
        cerr << "Не удалось загрузить Eps.dll" << endl;
        continue;
    }

    auto EpsFirst = (float (*)(int)) GetProcAddress(hLib, "EpsFirst");
    auto EpsSecond = (float (*)(int)) GetProcAddress(hLib, "EpsSecond");

    if (!EpsFirst || !EpsSecond) {
        cerr << "Ошибка загрузки функций из Eps.dll" << endl;
        FreeLibrary(hLib);
        continue;
    }

    int x;
    cout << "Введите значение числа x (int): ";
    cin >> x;

    float result = (switch_ == 1) ? EpsFirst(x) : EpsSecond(x);

```

```

        cout << "E = " << result << " - решен методом "
            << ((switch_ == 1) ? "(1 + 1/x)^x" : "суммой ряда по n от 0 до x, где
элементы равны: (1/(n!))") << endl;

        FreeLibrary(hLib);
    } else {
        cout << "Нет решения, так как не выбрана библиотека функции" <<
endl;
        break;
    }
}

return 0;
}

```

Задание 2

LinkingMain

```

#include <iostream>
#include <windows.h>
#include "Integral.h"
#include "Eps.h"
#include <string>

using namespace std;

int main(){

    cout << "Программа join на этапе линковки" << endl << endl;

    unsigned int switch_ = 1;

    while(1) {

        cout << "Введите название библиотеки, которую хотите
подключить: \n 'integral' - Интеграл \n 'eps' - Вычисление значения
натурального алгоритма \n" << endl;
        cout << "Введите Y, если хотите переключить реализацию
расчёта \n Изначальная реализация - 1" << endl;
        cout << "Введите 'exit' если хотети выйти из программы " <<
endl;

        string lib;
    }
}

```

```

string indicator;

if (!(cin >> lib) || !(cin >> indicator)) {
    cout << "Завершение работы программы" << endl;
    system("pause");
    break;
}
else if (indicator == "Y") {
    if (switch_ == 1) {
        switch_ = 2;
    }
    else {
        switch_ = 1;
    }
}
cout << "Реализация переключена на " << switch_ << endl;
}
else if (lib == "integral") {

    float a, b, eps;

    if (switch_ == 1) {
        cout << "Введите границы отрезка
интегрирования (a, b): " << endl;
        cin >> a >> b;

        cout << "Введите эпсилон: " << endl;
        cin >> eps;

        float integral = IntegrateFirst(a, b, eps);
        cout << "Ответ: " << integral << " - интеграл
решен методом прямоугольников" << endl;
        system("pause");
    }
    else {
        cout << "Введите границы отрезка
интегрирования (a, b): " << endl;
        cin >> a >> b;

        cout << "Введите эпсилон: " << endl;
        cin >> eps;

        float integral = IntegrateSecond(a, b, eps);
        cout << "Ответ: " << integral << " - интеграл
решен методом трапеций" << endl;
    }
}

```

```

        system("pause");
    }
}

else if (lib == "eps") {

    int x;

    if (switch_ == 1) {
        cout << "Введите значения числа x (int)" <<
endl;

        cin >> x;

        float eps = EpsFirst(x);
        cout << "E = " << eps << "- решен методом
(1+1/x)^x" << endl;

        system("pause");
    }
    else {
        cout << "Введите значение числа x (int) " <<
endl;

        cin >> x;

        float eps = EpsSecond(x);
        cout << "E = " << eps << "- решен методом
суммой ряда по n от 0 до x, где элементы равны: (1/(n!)) " << endl;
        system("pause");
    }
}
else {
    cout << "Нет решения, так как не выбрана библиотека
функции" << endl;

    system("pause");
    break;
}

return 0;
}

```

Вывод

В данной лабораторной работе были продемонстрированы два способа использования динамических библиотек: с линковкой на этапе компиляции и

на этапе выполнения. Основное отличие между ними — гибкость. Подключение на этапе выполнения позволяет менять реализацию функций по ходу работы программы, не требуя её перекомпиляции. Это даёт возможность создавать более гибкие приложения, особенно в случаях, когда есть необходимость динамически выбирать различные реализации функций или расширять функциональность программы.